

3D Gaussian Splatting Report

ACM Reference Format:

. 2025. 3D Gaussian Splatting Report. In . ACM, New York, NY, USA, 2 pages.
<https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

1 Introduction

In recent years, 3D Gaussian Splatting (3DGS)[2] has emerged as a highly efficient representation for neural scene reconstruction and rendering. Unlike traditional volumetric methods such as NeRFs, which rely on dense sampling along camera rays and computationally expensive volume rendering, 3DGS represents a scene as a collection of anisotropic Gaussian primitives that directly model radiance and opacity in 3D space. This representation allows for highly efficient differentiable rasterization, making 3DGS particularly suitable for real-time rendering scenarios. The ability to render photorealistic scenes interactively is of paramount importance for applications such as virtual and augmented reality (VR/AR), robotics perception, autonomous driving simulation, and digital twins. Real-time 3D reconstruction and rendering enable dynamic interaction with the virtual world, significantly enhancing both visual realism and user experience.

However, the challenge lies in the computational cost of training and optimizing such high-dimensional models. Even though 3DGS provides a more efficient representation compared to volumetric grids or implicit neural fields, training still involves iterative optimization over thousands of Gaussian parameters—each with attributes such as position, scale, orientation, color, and opacity. When high-resolution images are used as training input, the optimization process becomes memory-intensive and time-consuming. This restricts the scalability of 3DGS in real-world or online learning settings where rapid adaptation is essential. Consequently, achieving real-time or near real-time training and inference remains an open problem and a major research goal in this field.

To address these challenges, recent research has explored techniques such as hierarchical level-of-detail (LOD) optimization, adaptive sampling, and mixed-resolution rendering. However, most existing methods focus on accelerating the rendering stage, while the training process itself remains a bottleneck. Therefore, a strategy that accelerates convergence during training without compromising final rendering fidelity is critical to advancing 3DGS toward practical, real-time use.

2 Method

Our method for high-quality and fast 3D Gaussian Splatting (3DGS) reconstruction is designed to balance superior visual fidelity with

stringent latency constraints, targeting completion within 60 seconds as per the competition requirements. The core idea revolves around progressive resolution training and rapid fitting strategies, which enable efficient convergence without compromising reconstruction quality. Progressive resolution begins with low-resolution rendering to quickly capture global scene structure, gradually scaling up to full resolution for fine details. This approach minimizes computational overhead in early iterations by reducing pixel-wise operations, allowing the model to focus on coarse alignment before refining high-frequency elements. Coupled with rapid fitting, we leverage pre-existing SLAM data (e.g., initial point clouds and camera poses) to accelerate parameter optimization, bypassing traditional slow initialization phases and enabling faster adaptation to complex scenes like 360° object captures or partial coverage environments.

We build upon DashGaussian [1] as our baseline framework, which already incorporates dynamic scheduling for Gaussian growth and rendering resolution to achieve up to 45.7% speedup over vanilla 3DGS. DashGaussian’s adaptive densification and pruning mechanisms are particularly suited to our dataset, comprising a 1440x1680@30fps video, camera intrinsics (default 480x640 with aspect ratio preservation), extrinsic poses with timestamps, an initial point cloud, and frame-timestamp correspondences. To meet the 60-second latency bound while handling potential pose errors and scene variability, we made targeted modifications to enhance efficiency.

We adjusted the total iteration count, incorporating an early-stopping criterion to halt training once sufficient quality is reached. This reduction exploits the strong initial priors from the SLAM point cloud (points3D.txt), which we directly use for Gaussian initialization—mapping positions (X, Y, Z), colors (R, G, B), and scales (derived from ERROR) to Gaussian parameters. This skips random or SfM-based seeding and promoting rapid fitting.

Additionally, for the densification and pruning of 3DGS, we used VCD and VCP [3]. VCD (Multi-view Consistent Densification) is a strategy based on multi-view consistency, used to decide whether to add new Gaussian primitives to refine the 3D scene representation. Its principle involves randomly sampling multiple training views, rendering images, and comparing the pixel-level differences with the real images to identify high-error regions. Then, for each Gaussian, the average number of high-error pixels covered by its projected footprint in these views is evaluated; if the number exceeds a threshold, it indicates that the Gaussian makes a significant contribution to the multi-view rendering quality, and it is cloned or split to increase density, thereby avoiding the blind addition of redundant Gaussians. VCP (Multi-view Consistent Pruning), in contrast, complements this. Its principle is to calculate the negative impact of each Gaussian on the overall rendering error by integrating the photometric loss from multiple views and the high-error pixels within its footprint, thereby quantifying its contribution; if the contribution is low, the Gaussian is removed to eliminate redundancy. These two strategies work together in the training process of 3D Gaussian Splatting, strictly controlling the number of Gaussians,

Permission to make digital or hard copies of all or part of this work for personal or

Unpublished working draft. Not for distribution. This work is distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

Conference’17, Washington, DC, USA

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM

<https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

2025-11-22 11:42. Page 1 of 1-2.

significantly accelerating training and maintaining high-quality rendering without the need for complex budget mechanisms.

These optimizations were implemented in PyTorch training on CUDA-enabled GPUs to further accelerate forward/backward passes. The pipeline allocates 40-50 seconds for construction. Extensive benchmarking on the provided datasets confirmed our method achieves high PSNR/SSIM scores comparable to baselines while adhering to the time limit.

`\bibliographystyle{ACM-Reference-Format}`

`\bibliography{bibfile}`

References

- [1] Youyu Chen, Junjun Jiang, Kui Jiang, Xiao Tang, Zhihao Li, Xianming Liu, and Yinyu Nie. 2025. DashGaussian: Optimizing 3D Gaussian Splatting in 200 Seconds. In *CVPR*.
- [2] Bernhard Kerbl, Georgios Kopanas, Thomas Leimkühler, and George Drettakis. 2023. 3D Gaussian Splatting for Real-Time Radiance Field Rendering. *ACM Trans. Graph.* 42, 4 (2023), 139:1–139:14.
- [3] Shiwei Ren, Tianci Wen, Yongchun Fang, and Biao Lu. 2025. FastGS: Training 3D Gaussian Splatting in 100 Seconds. *arXiv preprint arXiv:2511.04283* (2025).